

Symplectic learning for Hamiltonian neural networks

Marco David^{a,b,*}, Florian Méhats^c

^a SpaceAble, Paris, France

^b École Normale Supérieure, PSL Research University, F-75005 Paris, France

^c Univ Rennes, CNRS, IRMAR – UMR 6625, F-35000 Rennes, France

ARTICLE INFO

Article history:

Received 4 March 2022

Received in revised form 22 March 2023

Accepted 7 September 2023

Available online 20 September 2023

Keywords:

Hamiltonian neural network

Ordinary differential equation

Hamiltonian system

Geometric numerical integration

Symplectic numerical method

ABSTRACT

Machine learning methods are widely used in the natural sciences to model and predict physical systems from observation data. Yet, they are often used as poorly understood “black boxes,” disregarding existing mathematical structure and invariants of the problem. Recently, the proposal of Hamiltonian Neural Networks (HNNs) took a first step towards a unified “gray box” approach, using physical insight to improve performance for Hamiltonian systems. In this paper, we explore a significantly improved training method for HNNs, exploiting the symplectic structure of Hamiltonian systems with a different loss function. This frees the loss from an artificial lower bound. We mathematically guarantee the existence of an exact Hamiltonian function which the HNN can learn. This allows us to prove and numerically analyze the errors made by HNNs which, in turn, renders them fully explainable. Finally, we present a novel post-training correction to obtain the true Hamiltonian only from discretized observation data, up to an arbitrary order.

© 2023 Elsevier Inc. All rights reserved.

1. Introduction

Machine learning models provide a promising new method to learn and predict the behavior of physical systems. With much scientific attention devoted to complex systems such as the climate [2,3], fluid dynamics [4,5], quantum mechanics [6], or space weather [7,8], machine learning might soon be a crucial tool for hybrid models [9], and maybe even replace some manually tuned models entirely [10].

Many physical models can be described using systems of differential equations. In particular, much general methodology to learn ordinary differential equations (ODEs) has recently been developed [11–14]. A specific and important class of physical models are Hamiltonian systems which are described by a single scalar function in phase space $H : \Omega \subseteq \mathbb{R}^{2n} \rightarrow \mathbb{R}$, often interpreted as the total energy of the system. The corresponding dynamics of the system for $y \in \Omega$ are described by Hamilton's equations [15,16]

$$\dot{y} = J^{-1} \nabla H(y), \quad J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad (1)$$

that is a system of ODEs with the particular vector field $J^{-1} \nabla H$. Here, J is the so-called *canonical symplectic matrix* and we also call the vector field the *symplectic derivative* of H . Solutions of this system are physical trajectories $y(t)$ which necessarily have the property that $H(y(t))$ stays constant at all times [16].

* Corresponding author at: École Normale Supérieure, PSL Research University, F-75005 Paris, France.

E-mail address: symplectic-hnn@ens.fr (M. David).

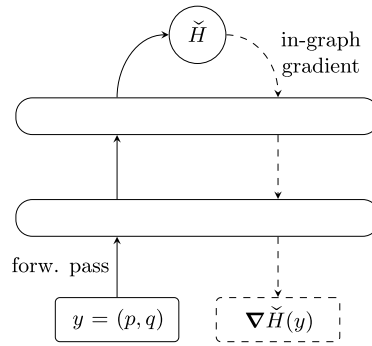


Fig. 1. Exemplary architecture of an HNN with two hidden layers, due to Greydanus et al. [1]. The loss function, which optimizes the gradient $\nabla \tilde{H}(y)$ of the neural network with respect to its input y , is the principal object of study in this article.

Greydanus et al. [1] have recently proposed a clever class of Hamiltonian Neural Networks (HNNs) whose architecture (see Fig. 1) engraves the mathematical properties of Hamilton's equations (notably their symplecticity). By asking the neural network to predict the Hamiltonian and then calculating its symplectic gradient using backpropagation, they were able to obtain trajectories in phase space that conserve the learned Hamiltonian over much longer periods of time than a baseline neural network which immediately learned the Hamiltonian vector field.

This work comprises three main contributions. In Section 2.1, we conduct a mathematical analysis of HNNs which exhibits an intrinsic obstacle that prevents them from learning an exact Hamiltonian function. This hampers the best attainable accuracy of HNNs. Reinterpreting the loss function as the result of a numerical integration scheme, this obstacle realizes as an artificial lower bound on the loss. In a novel yet elementary proof (see Appendix A), we show that this is due to the local error of the forward Euler integration method.

Section 2.2 then explores an improved training method for Hamiltonian Neural Networks which remedies this fact through a subtle change in the model's loss function. This approach has received several names but may simply be called *Symplectic Hamiltonian Neural Networks*, or SHNNs. We prove that there always exists an exact function, the so-called modified Hamiltonian, that SHNNs can learn to arbitrary precision. In conjunction with a thorough experimental verification of this result (see Section 3.2), we contribute to the explainability of general SHNNs.

Finally, Section 2.3 introduces an entirely new post-training correction that allows to obtain the true Hamiltonian up to an arbitrary desired order. Using SHNNs and only discretized data of the true solutions, this approach exploits known properties of the modified Hamiltonian. Concretely, a Hamilton-Jacobi partial differential equation (PDE) is used to obtain a formal series expansion of the true Hamiltonian which can be used to correct for the introduced discretization error.

1.1. Related work

Since the original proposal by Greydanus et al. [1] and the concurrent work by Bertalan et al. [17], HNNs have generated much scientific interest. To name a few, generative [18], recurrent [19] and constrained [20] versions, as well as Lagrangian Neural Networks [21] have been proposed.

Improvements of HNNs using symplectic integrators An adapted loss function for HNNs, derived from a symplectic numerical integration scheme, has been concurrently explored in several recent publications. Zhu et al. [22] use the implicit midpoint rule in an adapted loss and show that, in this case, the SHNN can learn an exact Hamiltonian given by the modified equation. Similarly, Chen et al. [19] make a modification using the leapfrog integrator before proposing a recurrent, multi-step training method, but only for separable Hamiltonians. Neither paper uses the modified equation to correct the learned Hamiltonian. Independently, Xiong et al. [23] use explicit higher-order symplectic methods during training, although they need to consider an augmented Hamiltonian on an augmented phase space with double the dimensionality. Finally, DiPietro et al. [24] use strong assumptions (including separability) to inform their architecture and train with a fourth-order symplectic integrator that is then explicit. However, they succeed in training with very small, sparse datasets.

Learning Hamiltonians from data using other architectures Beyond the immediate improvements of HNNs, there exist other proposals, notably other architectures, to learn Hamiltonians from data. Notably, Zhong et al. [20] learn both a strongly parametrized and a general Hamiltonian in conjunction with the Neural ODE [12] model. They also learn systems under the influence of external forces. Further, Jin et al. [25] introduce a new architecture to learn any symplectic map and prove respective approximation theorems. They use SHNNs with the implicit midpoint rule as their baseline. Tong et al. [26] also directly predict a future state of the system, but they use a separable Hamiltonian in conjunction with a then explicit fourth-order symplectic method. Finally, Chen and Tao [27] propose and analyze a method to exactly learn symplectic evolution maps which notably include general Hamiltonian dynamics.

2. Theory

Consider a physical system whose state is described by a position vector $q \in \mathbb{R}^n$ and a momentum vector $p \in \mathbb{R}^n$. Abbreviate this state as $y = (p, q) \in \Omega \subseteq \mathbb{R}^{2n}$. Written in position and momentum coordinates, Hamilton's equation (1) reads

$$\dot{p} = -\nabla_q H(p, q), \quad \dot{q} = \nabla_p H(p, q). \quad (2)$$

For example, in classical mechanics, the Hamiltonian can be written as the sum of kinetic and potential energy leading to a so-called separable Hamiltonian $H(p, q) = T(p) + V(q)$. Further, the kinetic energy of classical massive particles is $T(p) = p^2/2m$ such that Hamilton's equation reduces to Newton's second law for the acceleration $\ddot{q}$:

$$\dot{q} = \frac{p}{m} \quad \text{and} \quad \dot{p} = -\nabla V(q) \implies m\ddot{q} = -\nabla V(q). \quad (3)$$

If the Hamiltonian is known, the system (1) can be numerically integrated using a *numerical integration method* Φ_h that maps $\hat{y}_i \mapsto \hat{y}_{i+1} = \Phi_h(\hat{y}_i)$, where h is the integration time step. Let $\varphi_t(y)$ denote the true flow of the system starting from the initial position y at time $t = 0$. The *order* of a numerical method is then defined to be the largest exponent r such that $\Phi_h(y) - \varphi_h(y) = \mathcal{O}(h^{r+1})$.

For example, the simplest and most naive numerical method, the forward Euler method, simply updates $\hat{y}_i$ by h times the ODE vector field evaluated at that point: $\Phi_h(\hat{y}_i) = \hat{y}_i + hJ^{-1}\nabla H(\hat{y}_i)$. In position and momentum coordinates, this first-order method explicitly becomes

$$\hat{p}_{i+1} = \hat{p}_i - h\nabla_q H(\hat{p}_i, \hat{q}_i) \quad \text{and} \quad \hat{q}_{i+1} = \hat{q}_i + h\nabla_p H(\hat{p}_i, \hat{q}_i). \quad (4)$$

A much more useful class of numerical methods are so-called *symplectic methods*, defined by the fact that $\Phi_h(y)$ is a symplectic mapping (i.e. its Jacobian matrix $\Phi'_h(y)$ leaves the canonical symplectic form J invariant). In two dimensions, these transformations preserve the area in phase space. Further, they preserve a scalar quantity close to the real Hamiltonian, called the *modified Hamiltonian* (see "Backward Error Analysis" [28, Ch. IX] which may be called "conjugate" to the below analysis). Although symplectic methods also introduce an error of order h^{r+1} at each step, due to their geometric nature, they are much better suited to integrate Hamilton's equation.

2.1. Hamiltonian neural networks

In contrast to obtaining trajectories from a known Hamiltonian, the purpose of Hamiltonian Neural Networks (HNNs) [1] is to learn a Hamiltonian from data, composed of observed trajectories $y(t)$ which solve Hamilton's equation (1). More specifically, we consider a data point to be a couple $(y_0, y_1 = \varphi_h(y_0))$ of two consecutive snapshots of the state of the system separated by a time $\Delta t = h$. Having two data points is crucial to have information about the evolution of the system and to calculate a finite difference approximation $(y_1 - y_0)/h$ of the time derivative $\dot{y}$. Since any trajectory with n data points can be split into $n - 1$ such couples, we shall consider all our data to be in this form.

Denote the HNN by the function $\check{H}(p, q)$ which implicitly also depends on all weights and biases of the chosen neural network, see Fig. 1 for a sketch of the architecture of HNNs. In the spirit of Greydanus et al. [1], its loss function¹ for one data point (y_0, y_1) is

$$\mathcal{L}_{\text{HNN}} = \left\| \frac{y_1 - y_0}{h} - J^{-1}\nabla\check{H}(y_0) \right\|_{L^2}^2 \quad (5)$$

which we shall rewrite into the form

$$\mathcal{L}_{\text{HNN}} = h^{-2} \left\| y_1 - \underbrace{(y_0 + hJ^{-1}\nabla\check{H}(y_0))}_{=\hat{y}_1} \right\|_{L^2}^2. \quad (6)$$

Now it becomes clear that, up to a constant factor of h^2 , we are effectively integrating the HNN's prediction $\check{H}$ using the forward Euler method and comparing the result $\hat{y}_1$ to the real observation y_1 in the loss function. We can do better than forward Euler!

Remarks.

- (i) In fact, using the forward Euler method here means that there does not even exist a function $\check{H}$ such that the loss could be identically zero everywhere in phase space; a consequence of the converse Poincaré theorem [28, Thm. VI.2.6]. This impossibility realizes as a mismatch of order h in the mixed second derivatives $\nabla_{pq}\check{H}$ and $\nabla_{qp}\check{H}$ of the function

¹ Note that Greydanus et al. [1] used the analytic gradient of the true Hamiltonian as the target for most tasks, which yields a different mathematical problem, i.e. learning a known scalar function from its gradient. The present article only uses finite differences as would be obtained from real data.

to be learned, as explained in Appendix A. An artificial lower bound for the loss is thus introduced, frustrating the training procedure.

- (ii) One may argue that, instead, the true derivative $\dot{y}$ should be more accurately approximated by higher-order difference quotients. Yet, this would only involve operations on the dataset without the HNN itself, so this point of view is rather limited. One would not be able to exploit the real system's symplecticity as explained next.

2.2. Symplectic methods to the rescue

According to the theory of geometric numerical integration [28], the forward Euler method should be replaced by a symplectic method. The two simplest symplectic methods are the *symplectic Euler method*

$$p_1 = p_0 - h \nabla_q H(p_1, q_0), \quad q_1 = q_0 + h \nabla_p H(p_1, q_0) \quad (7)$$

and the *implicit midpoint rule*

$$y_1 = y_0 + h J^{-1} \nabla H\left(\frac{y_0 + y_1}{2}\right). \quad (8)$$

Both methods are implicit and are obtained by only changing the point of evaluation of the Hamiltonian vector field. Abstracting the choice of a specific integration scheme by a function $s(y_0, y_1)$ (this covers all methods of interest in this article), we obtain the loss function for Symplectic Hamiltonian Neural Networks (SHNNs):

$$\mathcal{L}_{\text{SHNN}} = \left\| \frac{y_1 - y_0}{h} - J^{-1} \nabla \tilde{H}(s(y_0, y_1)) \right\|_{L^2}^2 \quad (9)$$

(provided that $s(y_0, y_1)$ gives rise to a symplectic integration method)

Using a symplectic method ensures that there exists a function $\tilde{H}$, which the SHNN can theoretically learn to an arbitrary precision (assuming that the neural network is large enough and that the dataset sufficiently covers the input space), as encapsulated by the following proposition. We shall henceforth refer to $\tilde{H}$ as the *(forward) modified Hamiltonian*. Note that the solutions of a general Hamiltonian system are not necessarily well-behaved for long times, so all following theoretical results are non-trivial and meaningful only for not too large h .

Proposition 1. Let $H : \Omega \subseteq \mathbb{R}^{2n} \rightarrow \mathbb{R}$ be a smooth Hamiltonian and $\dot{y} = J^{-1} \nabla H(y)$ be the corresponding Hamiltonian system. Fix $h > 0$. Denote the true flow of the system after time h by φ_h , and write $\varphi_h(y_0) =: y_1 = (p_1, q_1)$ wherever this is well-defined for some initial condition $y_0 = (p_0, q_0)$. Then,

- (a) (symplectic Euler) there exists a locally-defined smooth function $\tilde{H}_{se}$ such that

$$J^{-1} \nabla \tilde{H}_{se}(p_1, q_0) = \frac{y_1 - y_0}{h} \quad (10)$$

for all $y_0 \in \Omega$ where $y_1 = \varphi_h(y_0)$ is well-defined.

- (b) (implicit midpoint) there exists a locally-defined smooth function $\tilde{H}_{mp}$ such that

$$J^{-1} \nabla \tilde{H}_{mp}\left(\frac{y_1 + y_0}{2}\right) = \frac{y_1 - y_0}{h} \quad (11)$$

for all $y_0 \in \Omega$ where $y_1 = \varphi_h(y_0)$ is well-defined.

Proof. This is exactly the result of Lemma 5.3 in Hairer et al. [28, Ch. VI], using $\tilde{H}_{se} = \frac{1}{h} S^1$ or $\tilde{H}_{mp} = \frac{1}{h} S^3$ and noting that φ_h is a smooth symplectic transformation close to the identity. $\square$

Remarks.

- (i) A more general result of existence for any general symplectic integration method is given by Chartier et al. [29, Sec. 2.2] using so-called B-series, a generalized, formal Taylor expansion. What they call the “modified differential equation,” written with a modified Hamiltonian, is exactly what an SHNN learns.
- (ii) Note that when numerically solving ODEs, implicit methods like the symplectic Euler method and midpoint rule normally require fixed point iterations at each step. However, during the training of HNNs we are solving the reverse problem for which the true trajectories (y_0, y_1) are already known. This means that symplectic training with implicit methods has the same computational cost as training with explicit methods.
- (iii) One property of the learned Hamiltonian should be noted: *Integrating the respective Hamilton equation of the modified $\tilde{H}$ with the same method and same time step h as used during training, one will obtain the true flow φ_h^h of the real Hamiltonian.* This is the statement of Theorems 5.7 and 5.8 of [28, Ch. VI] (noting that the necessary assumptions are satisfied due to Proposition 2 below). However, there is no free lunch: The true intermediate states of the system will not be accessible nor predictable with this method. In fact, reducing the integration time step in this case will worsen the quality of the trajectory.

2.3. Correction of the learned Hamiltonian

What's more, we can derive a formal series for the modified Hamiltonian $\check{H}$ learned by an SHNN in terms of the real Hamiltonian H and its derivatives, depending on the used integration method. This allows us to not only understand exactly what our model learns and where it draws its predictive power from, but also to correct the model after training to an arbitrary order. This way, we can learn the real, physical Hamiltonian $H(p, q)$ to arbitrary precision, purely from discretized snapshots of trajectory data, without any information about the true gradients or vector fields.

Mathematically, Hamilton's equation is the characteristic equation of the Hamilton-Jacobi partial differential equation (PDE), obtained by considering the flow of the true Hamiltonian system after a variable time $t \in \mathbb{R}$. Rendering explicit the fact that the modified Hamiltonian naturally depends on the time step h fixed in Proposition 1, we write $\check{H}(p, q, t = h)$. Since the flow is a smooth function of time when it exists, $\check{H}$ will also be smooth in all its variables. Calculating its time derivative, this leads to the following.

Proposition 2. *Let $H : \Omega \subseteq \mathbb{R}^{2n} \rightarrow \mathbb{R}$ be a smooth Hamiltonian. For both cases of Proposition 1, there exists a neighborhood of $t = 0$ where the respective time-dependent modified Hamiltonian $\check{H}(p, q, t)$ solves a Hamilton-Jacobi PDE. Explicitly,*

(a) (symplectic Euler) there exists an open neighborhood $U \ni 0$ such that $\forall t \in U$,

$$\frac{\partial}{\partial t}(\check{H}_{se}(p, q, t)) = H\left(p, q + t \frac{\partial \check{H}_{se}}{\partial p}(p, q, t)\right) \quad (12)$$

(b) (implicit midpoint) there exists an open neighborhood $U \ni 0$ such that $\forall t \in U$,

$$\frac{\partial}{\partial t}(\check{H}_{mp}(y, t)) = H\left(y + \frac{t}{2} J^{-1} \nabla_y \check{H}_{mp}(y, t)\right) \quad (13)$$

Integrating either of these relations from 0 to $h \in U$ and Taylor expanding the right-hand side generates a formal power series in h whose coefficients depend on H and its (partial) derivatives.

Proof. This is a direct calculation using generating functions [28, Sec. VI.5.3]. $\square$

This proposition allows the computation of $\check{H}$ learned by an SHNN if the Hamiltonian H of the original problem is known (see also [28, Sec. VI.5.4]). However, in practice, we will want to reconstruct H from the modified $\check{H}$ learned from data. Hence, the formal power series needs to be inverted to the desired order. Since the series is purely formal, this can be easily done with the help of symbolic computation programs. For the symplectic Euler method, abbreviating $H = H(p, q)$ and $\check{H} = \check{H}_{se}(p, q, h)$, this yields

$$\begin{aligned} H = \check{H} - \frac{h}{2} \nabla_p \check{H} \cdot \nabla_q \check{H} \\ + \frac{h^2}{12} \left(\nabla_{pp} \check{H} (\nabla_q \check{H})^2 + 4 \nabla_{pq} \check{H} (\nabla_p \check{H}, \nabla_q \check{H}) + \nabla_{qq} \check{H} (\nabla_p \check{H})^2 \right) + \mathcal{O}(h^3). \end{aligned} \quad (14)$$

Similarly, for the implicit midpoint method, one obtains

$$H = \check{H} - \frac{h^2}{24} \nabla^2 \check{H} (J^{-1} \nabla \check{H}, J^{-1} \nabla \check{H}) + \mathcal{O}(h^4). \quad (15)$$

Both of these equations are precisely the standard series which express the (backward) modified Hamiltonian of a Hamiltonian system via Backward Error Analysis. This calculation renders explicit how our analysis of SHNNs is conjugate to this well-known method: The backward modified Hamiltonian of the learnt (forward modified) Hamiltonian $\check{H}$ is again the true Hamiltonian H . We shall refer to terms from these series as *corrections* to the learnt Hamiltonian, reducing the error caused by our discrete data.

Aside from the question which function can be learned in principle, one may ask how well a given network architecture approximates the target $\check{H}$. As one example of recent theoretical work on this question (see also [25,30]), De Ryck et al. [31] have studied fully connected neural networks with two hidden layers and a tanh activation function.² The following corollary uses their results on the universal approximation of Sobolev functions by such an architecture.

² This makes the neural network smooth and also exactly matches the architecture used for most of our numerical experiments.

Corollary. Let $H : \Omega \subseteq \mathbb{R}^{2n} \rightarrow \mathbb{R}$ be a smooth Hamiltonian system. Then, a SHNN $\tilde{H}$ with two hidden layers of M neurons each and a tanh activation function, trained with a symplectic method of order r and hyperparameters $\vec{\beta}$ on K data points separated by a time step h , makes the following error on a compact smooth region $\Omega_m \subset \Omega$. For every $s \in \mathbb{N}$, it holds that

$$\|\tilde{H} - H\|_{L^\infty(\Omega_m)} \leq C_1(r, \tilde{H})h^r + C_2(n, s, \tilde{H})M^{-s} + \delta(K, M, \vec{\beta}, \dots). \quad (16)$$

Remark. The constants C_1 and C_2 depend, respectively, on the $W^{r,\infty}(\Omega_m)$ and $W^{s,\infty}(\Omega_m)$ Sobolev norms of the modified Hamiltonian.³

The corollary is proven with the triangle inequality. The first term captures the error between the modified and true Hamiltonians $\tilde{H}$ and H as developed earlier in this section. The second term captures the error in approximating Sobolev functions by a fully-connected tanh neural network which consists of iterated instances of $\tanh(Ax+b)$, following the theory developed by De Ryck et al. [31]. The third term captures the learning error between the best possible approximation of $\tilde{H}$ and the real network $\tilde{H}$, due to the data set, the chosen optimizer and its hyperparameters. The following section operationalizes this result.

3. Numerical experiments

Using a numerical implementation⁴ of SHNNs, we have tested all of the above theory on three tasks of varying difficulty. The first two tasks were also treated by Greydanus et al. [1] in the original proposal of HNNs.

Task 1: the harmonic oscillator The “Hello World” of Hamiltonian systems is the harmonic oscillator with non-dimensionalized Hamiltonian $H(p, q) = \frac{1}{2}p^2 + \frac{1}{2}q^2$. This simple system is of fundamental importance in physics as it is described by a quadratic potential (linear force). It models e.g. a mass on a spring, where q is the displacement from equilibrium and p the momentum.

Task 2: the non-linear pendulum Another important system is the non-linear ideal pendulum with non-dimensionalized Hamiltonian $H(p, q) = \frac{1}{2}p^2 + (1 - \cos q)$. Instead of a linear restoring force, the gravitational force is now proportional to $\sin q$, where q models the angle of the pendulum measured from equilibrium and p consequently models the angular momentum.

Task 3: the double pendulum As a non-separable system of dimension 4 with chaotic dynamics, the double pendulum presents the most challenging task. Its non-dimensionalized Hamiltonian

$$H(p_1, p_2, q_1, q_2) = \frac{\frac{1}{2}p_1^2 + p_2^2 - p_1p_2\cos(q_1 - q_2)}{1 + \sin^2(q_1 - q_2)} - 2\cos q_1 - \cos q_2 \quad (17)$$

describes two equal masses at angles $q_{1,2}$ and angular momenta $p_{1,2}$. The first mass is attached to a fixed point, and the second mass is attached to the first, using stiff massless rods of equal length.

3.1. Methods

For each task, four different models were trained based on the loss function (9), for different choices of the scheme function $s = s(y_0, y_1)$ and post-training correction: forward Euler, symplectic Euler, implicit midpoint, and corrected symplectic Euler.

The forward Euler scheme $s = y_0$ replicates HNNs [1] trained with discretized data and represents our baseline. The symplectic Euler scheme $s = (p_1, q_0)$ is a symplectic method of order 1 whereas the implicit midpoint rule $s = (y_0 + y_1)/2$ is a symplectic method of order 2. Finally, we also trained an SHNN with the symplectic Euler method but afterwards corrected its Hamiltonian using $\tilde{H} - \frac{h}{2} \nabla_p \tilde{H} \cdot \nabla_q \tilde{H}$, obtaining a Hamiltonian correct up to second order.

For each task, we defined a bounded subregion Ω_d of the full phase space Ω to generate the data from. Given a fixed time step $h > 0$, we generated a dataset of K data points. Each point is given by a couple (y_0, y_1) where y_0 is a random initial state chosen uniformly from Ω_d and $y_1 = \varphi_h(y_0)$ represents a snapshot of the system's true solution at a time h later. Note that friction was neglected for all tasks; in fact, the architecture of HNNs prevents them from learning any change of the total energy with time. The full data set was separated using a test split of 20%.

We used fully connected neural networks with a tanh activation function, L hidden layers and M neurons per hidden layer for all tasks. All models were trained with the AdamW optimizer [32,33] as implemented in PyTorch [34] using default

³ Moreover, note that the network is only trained on derivatives of its output, hence it is able to learn $\tilde{H}$ solely up to some global bias, as corrected for later in eq. (18). We assume this global bias to be zero for the corollary above to make sense.

⁴ Our source code, synthetic datasets and pre-trained models are openly available at: <https://github.com/SpaceAbleOrg/symplectic-hnn>.

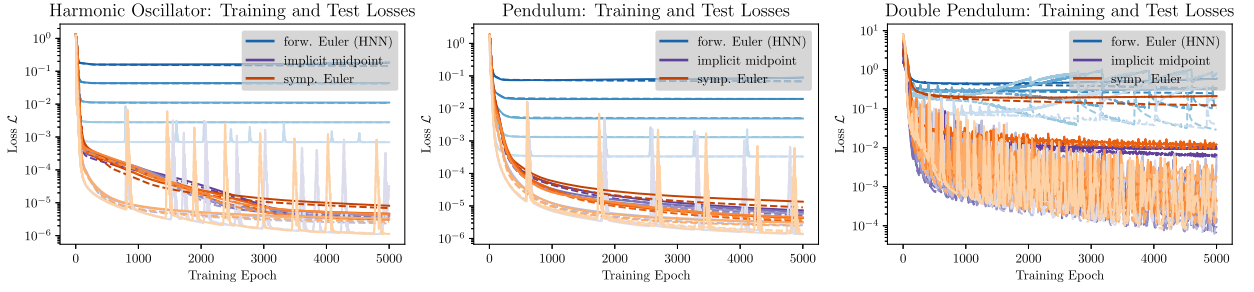


Fig. 2. Training (dashed lines) and testing (solid lines) losses as a function of the training epoch for the three chosen tasks, the different integration methods and different discretization time steps $h \in \{0.05, 0.1, 0.2, 0.4, 0.8\}$. For each method, the darkest shade of its color corresponds to the largest $h = 0.8$ and the lightest shade to the smallest $h = 0.05$.

coefficients, a learning rate of 10^{-3} and weight decay of 10^{-2} , for 5000 epochs without mini-batches (i.e. batch size = K). Only the model with the best test loss was saved after training. Table B.1 in Appendix B summarizes the different choices of model and dataset size for all tasks. Training was performed on a single GPU (Nvidia Tesla K80, cloud hosted) using the CUDA framework version 11.2 [35] as integrated in PyTorch [34].

The test and train L^2 losses were tracked per epoch while training the models. Afterwards, we measured our principal metric: the average error ε_H of the learned Hamiltonian over a region $\Omega_m \subset \Omega_d$ of phase space. This quantity was measured as

$$\varepsilon_H = \left\langle \left| \tilde{\mathcal{H}} - H - \langle \tilde{\mathcal{H}} - H \rangle_{\Omega_m} \right| \right\rangle_{\Omega_m}, \quad (18)$$

where the mean difference between $\tilde{\mathcal{H}}$ and H is removed inside the absolute value because the neural network only learns (an approximation of) $\tilde{\mathcal{H}}$ up to some global constant. Intuitively, one would like to remove this constant by evaluation at a single point as $\tilde{\mathcal{H}} - H - (\tilde{\mathcal{H}}(y_*) - H(y_*))$. However, this would add the local error at $y = y_*$ to the function everywhere, whereas the mean does not suffer from any locality issues. In particular, this measure $\varepsilon_H \leq \|\tilde{\mathcal{H}} - H\|_{L^\infty(\Omega_m)}$ as estimated in our Corollary in Section 2.

As an additional metric, we roll out long-term predictions of the trained models from random initial points, using the explicit Runge-Kutta method Dormand and Prince of order 5(4) [36] implemented in the `scipy.integrate` module [37]. Those trajectories are analyzed in two different fashions. Analyzing the shape of their trajectories in phase space, following the level curves of the modified Hamiltonian, provides insight into this modification with respect to the true Hamiltonian. Independently, measuring the mean squared L^2 error (MSE) between the long-term predictions of our models and the true solution provides insight into the quality of the predictions.

Remark. The measuring region Ω_m was chosen as the hypercube centered and contained within the data region $\Omega_d \subseteq \mathbb{R}^{2n}$, with side lengths divided by $\sqrt{2}$. The average error was not measured directly on Ω_d because our models perform drastically worse close to the boundary of Ω_d (see Appendix C).

3.2. Results

The SHNN models trained well on all datasets, with only minimal overfitting. Fig. 2 shows the training and test loss as a function of the training epoch, for an HNN trained with the forward Euler scheme and two SHNNs trained with the symplectic Euler and implicit midpoint schemes, respectively. Remarkable is the fact that the HNN losses plateau very quickly and, depending on the chosen time step h , do not descend below a certain threshold. These lower bounds of the squared L^2 loss are proportional to h^2 which confirms the theoretical result of Appendix A. The fact that they vary by a constant factor realizes as a constant difference on the logarithmic scale. Contrarily, for the trained SHNNs and all three tasks, the losses descend independently of h down to a level of numerical accuracy.

Further, we analyzed the average error of the learned Hamiltonian, which allows to draw inferences on the order of the used numerical method. The averages of equation (18) were calculated using $N = 2000$ points uniformly drawn from Ω_m . The mean and quartiles are shown in Fig. 3 on a double logarithmic scale, which means that an error of order h^p realizes as a straight line with slope p . This figure shows that using the forward or symplectic Euler methods yields an error of order h , and that using the implicit midpoint method yields an error of order h^2 as expected. Further, they confirm that the post-training correction to the SHNN trained with the symplectic Euler method also yields an error of order h^2 .

Finally, Fig. 4 shows exemplary long-term trajectories for the spring (Task 1) and non-linear pendulum (Task 2) and a large step $h = 0.8$, which makes the differences between the used integration schemes well visible. Several observations can be made. First, while the true Hamiltonian vector field is in general well learned by the SHNN, relatively large errors are visible at the edge of Ω_d . Second, training with the implicit midpoint rule later predicts the most accurate trajectories, followed by the corrected symplectic Euler method. This is the case both in the shape of the trajectory as well as in the

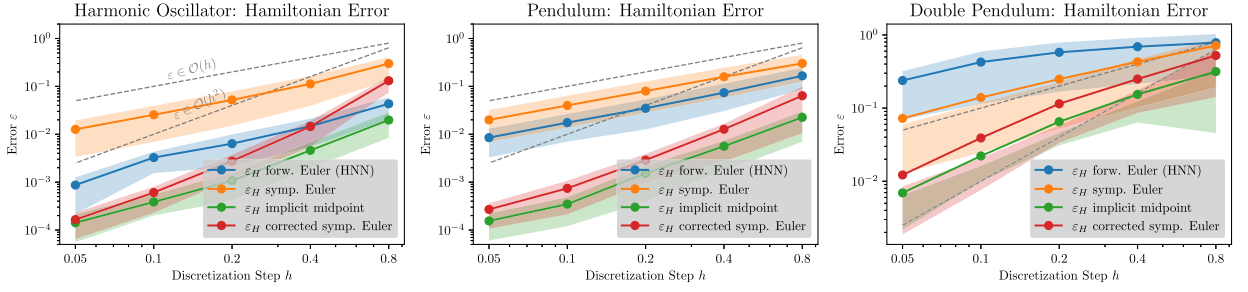


Fig. 3. Average error of the learned Hamiltonian ε_H as a function of the discretization time step h , for the three chosen tasks and the different integration methods ($N = 2000$). For each point the mean (solid marker) and quartiles (transparent region) are plotted, i.e. 50% of all data points respectively lie inside the colored transparent regions. Note that, for every point, the standard error of the mean is too small to be visible. Two reference lines $\varepsilon = h$ and $\varepsilon = h^2$ have been added in gray.

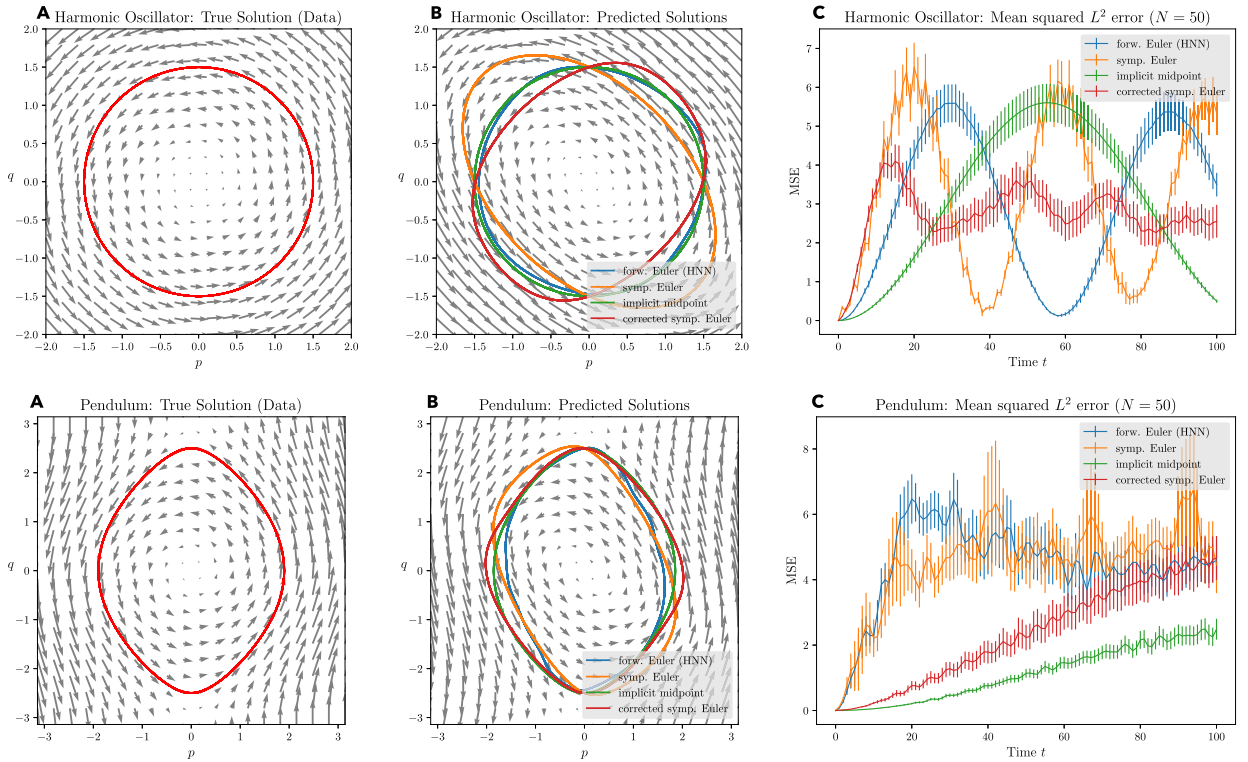


Fig. 4. Analysis of individual trajectories of the pendulum and harmonic oscillator, with different SHNNs, trained with a discretization step $h = 0.8$. The explicit trajectories drawn in phase space started at $y_0 = (0, 1.5)$ for the harmonic oscillator and $y_0 = (0, 2.5 \text{ rad})$ for the pendulum. The displayed regions of phase space are Ω_d in each case. **A:** True vector field and solution. **B:** Vector field as learned by an SHNN (trained with symplectic Euler) and predicted trajectories with all methods. **C:** Long-time results. MSE and standard error of the mean of $N = 50$ trajectories, with initial points drawn randomly from Ω_m , under the additional restriction that the system does not have enough energy to leave Ω_d ; since our models were not trained outside of Ω_d .

long-term MSE. Third, SHNNs with the symplectic Euler method learn a highly eccentric shape in phase space. This reflects the asymmetry of the method due to the evaluations at $s = (p_1, q_0)$, and in the case of the harmonic oscillator, this explains the perfect ellipse that results. Contrarily, the symmetric implicit midpoint method does not show this behavior. Fourth, correcting after training with symplectic Euler does improve the result (especially in the MSE) but “overshoots” the goal in phase space – the corrected ellipse is eccentric in the opposite direction, as expected from the alternating signs in equation (14).

It is to be noted that the baseline HNN performs exceptionally well on Task 1 due to its simplicity. However, as Fig. 3 shows, too, the more complex the model, the worse the performance of an HNN trained with the forward Euler method. We shall also point to Appendix D which allows to better understand the oscillations of the MSE.

4. Discussion and conclusion

The experimental results presented above confirm the theory developed in Section 2. The loss curves of Fig. 2 experimentally show that there is no exact function which can be learned by an HNN due to a mismatch depending on the chosen discretization step h . The errors of the learned Hamiltonians plotted as a function of h in Fig. 3 demonstrate first that HNNs inherit their order from the numerical method used during training. Second, they show that post-training corrections indeed allow one to obtain higher orders by exploiting the formal series expansion obtained from the Hamilton-Jacobi PDEs in Proposition 2.

This last point deserves further discussion. Up to our knowledge, for general (i.e. non-separable) Hamiltonian systems, there do not exist other methods that allow a non-recurrent SHNN to efficiently learn the modified Hamiltonian accurately to any order larger than 2. If the dataset is of the form $\{(y_0, y_1)_i\}$, symplectic methods of higher orders are generally implicit multi-stage methods,⁵ i.e. they use intermediate points like $y_{1/2}$ which are only implicitly determined themselves (unless the Hamiltonian is separable). Training with such a method means solving an implicit equation determined by the model for each data point and afterwards calculating the gradient of this operation – a huge computational cost! In contrast, our post-training method only makes predictions slightly more expensive. Per time step, the computational cost is larger by a constant factor due to the extra evaluations of the model's derivatives, according to the correction formulas (14) and (15). Better yet, one may even hard-code any given correction to a model once trained.

Finally, we wish to note some observations for the double pendulum system. In Figs. 2 and 3, the loss curves and average errors for $h = 0.4$ and $h = 0.8$ do not follow the theoretical expectation. We suspect that this is because the coarse grained data for these step sizes does not fully capture the (chaotic) dynamics of the system, and hence prevents accurate training. Additionally, for all values of h , the fact that the mean error in Fig. 3 lies outside the middle two quartiles shows that the error distribution is highly skewed, even after restriction to Ω_m .

4.1. Limitations

Real world data is never perfect. The principal limitation of the present, theory-guided article is the fact that it does not yet account for noisy data, which will deteriorate the quality of the learned Hamiltonian. Before SHNNs can be used to extract the behavior of real physical systems (see below), this effect will need to be quantified.

Further, since we are learning a continuous function with a neural network, the dataset has to densely cover the relevant region in the input space (phase space) to obtain a high-quality model. Such dense and vast datasets may not be available in reality. Yet, especially for high-dimensional systems, restricting to small regions where data is available does not inhibit solid results, also when these regions have holes, or even when considering multiple disconnected components.

4.2. Outlook

Using symplectic training and corrections of the modified Hamiltonian makes HNNs more powerful, but many further generalizations of our method could be considered. For example, directly learning the time-dependent generating functions $H(p, q, t)$ of Proposition 2 from a data set $\{(y_0, t_0, y_1, t_1)_i\}$ with variable time steps is an interesting question for future research. Alternatively, generalizations to Poisson systems $\dot{y} = B(y)\nabla H(y)$ (with suitable conditions on the matrix $B(y)$), which model e.g. interactions with electromagnetism or allow to express Hamiltonian mechanics in non-canonical coordinates [28, Sec. VII.2], seem like another fruitful subject.

In conclusion, Symplectic Hamiltonian Neural Networks are a promising “gray-box” approach, using physics-priors to build better machine learning algorithms and simultaneously explain why they work. Applications to almost all fields of physics are imaginable, and seem especially exciting in data-rich yet hard-to-model disciplines like the earth's climate or space weather.

CRedit authorship contribution statement

Marco David: Data curation, Formal analysis, Software, Writing – original draft. **Florian Méhats:** Conceptualization, Formal analysis, Resources, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Marco David reports financial support and equipment, drugs, or supplies were provided by SpaceAble. Florian Mehats reports a relationship with SpaceAble that includes: consulting or advisory.

⁵ Explicit symplectic methods for non-separable Hamiltonians do exist, too, although they require an augmented phase space [38] twice the size of the physical phase space. For these methods, it is a priori not clear that an exact modified Hamiltonian exists in physical phase space.

Data availability

Data and source code are openly available and linked in the article.

Acknowledgments and Disclosure of Funding

The authors would like to express their deep gratitude to SpaceAble for sponsoring, supporting and encouraging this project. In particular, thank you to Issao Ueda, Arnaud Bellizzi, Louis Celier, Quentin Gueho and Julien Cantegreil for all their help. The authors further thank Philippe Chartier for interesting discussions and pointers in the right directions.

Appendix A. Non-existence of a learnable Hamiltonian $\tilde{H}$ for HNNs

Consider a real smooth Hamiltonian $H : \Omega \rightarrow \mathbb{R}$ and let $y(t) = (p(t), q(t))$ denote the smooth exact solution of the corresponding Hamilton equation. Hamiltonian Neural Networks as introduced by Greydanus et al. [1] are asked to predict a smooth scalar function $\tilde{H} : \Omega \rightarrow \mathbb{R}$. The optimization, however, uses its symplectic in-graph gradient $J^{-1}\nabla\tilde{H}$ (5) and the numerical forward Euler method. As this method is not symplectic, the corresponding ODE which the neural network tries to learn is not Hamiltonian [28, Thm. VI.2.6]. We can quantify the extent to which the learned ODE is not Hamiltonian, which we expect to realize in the loss during training.

Concretely, fixing $h > 0$ such that an observation $(y_0, y_1) = (y(0), y(h))$ exists, we ask that the network learn $\tilde{H}$ with the following loss⁶

$$-\nabla_q \tilde{H}(p_0, q_0) \stackrel{L}{=} \frac{p_1 - p_0}{h} \quad \text{and} \quad \nabla_p \tilde{H}(p_0, q_0) \stackrel{L}{=} \frac{q_1 - q_0}{h}. \quad (\text{A.1})$$

After Taylor expanding y_1 around y_0 and using Hamilton's equations, one obtains the following for the mixed second derivatives of the network:

$$\nabla_{qp} \tilde{H} - \nabla_{pq} \tilde{H} \stackrel{L}{=} h (\nabla_{pq} H \cdot \nabla_{qp} H - \nabla_{pp} H \cdot \nabla_{qq} H) \neq 0. \quad (\text{A.2})$$

Noting that any neural network with fully connected linear layers and a smooth activation function (like tanh in our case) is also smooth, partial derivatives should commute and this difference should always be zero. However, this analysis tells us that standard HNNs will learn a function whose gradients match those of H only up to an error (a mismatch) of order h . The squared L^2 loss of eq. (5) will thus be proportional to h^2 when using the forward Euler method.

Appendix B. Model sizes and corresponding hyperparameters

For each task and discretization time step h , we generated a dataset of K points composed of two snapshots (y_0, y_1) of the system, taken a time $\Delta t = h$ apart from each other. Further, we specified the size of a neural network with L hidden layers of M neurons, respectively, and then trained several models using the three different numerical integration schemes (forward Euler, symplectic Euler, implicit midpoint). The following table summarizes these choices.

Table B.1

Summary of model and dataset parameters for each task and time step h . Each cell specifies a network of L hidden layers with M neurons each, and a dataset of K points, in the format $L, M; K$.

	$h = 0.8, 0.4, 0.2$	$h = 0.1$	$h = 0.05$
Spring ($\Omega_d = [-1, 1]^2$)	1, 200; 2k	2, 200; 4k	3, 200; 10k
Pendulum ($\Omega_d = [-\pi, \pi]^2$)	1, 200; 2k	2, 200; 4k	3, 200; 10k
Double Pend. ($\Omega_d = [-\pi, \pi]^4$)	2, 400; 100k	3, 600; 100k	3, 600; 100k

Appendix C. Analysis of the Hamiltonian error distribution in Ω_d

An analysis of the error distribution $\varepsilon_H(p, q)$ in phase space Ω_d proves very insightful (see Fig. C.5). The below error distributions reveal a mixture of the shape of the true Hamiltonian as well as the geometric properties of the used numerical integration methods. Additionally, these distributions also confirm that the error is proportional to h^p . Most importantly, however, they show that the error becomes very large at the edge of Ω_d which motivates our choice of a restricted measuring region $\Omega_m \subsetneq \Omega_d$.

⁶ For this analysis, it is key to consider finite differences on the right-hand side. We do not treat the case where these differences are replaced by true analytic gradients of H .

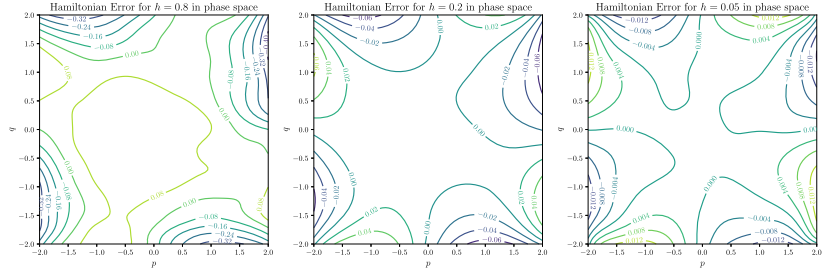
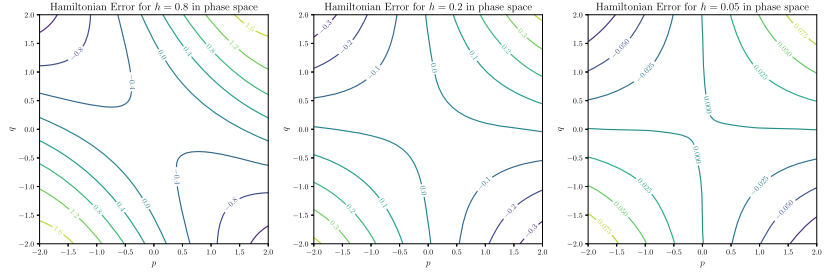
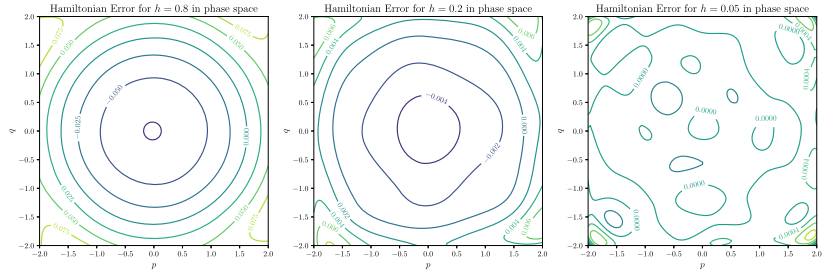
(a) Harmonic Oscillator, forward Euler (HNN), $h = 0.8, 0.2, 0.05$ (b) Harmonic Oscillator, symplectic Euler, $h = 0.8, 0.2, 0.05$ (c) Harmonic Oscillator, implicit midpoint, $h = 0.8, 0.2, 0.05$ 

Fig. C.5. Distribution of the error of the Hamiltonian $\tilde{H}$ learned by an SHNN with respect to the true known Hamiltonian H . The different models were trained for the harmonic oscillator (Task 1) with the forward Euler, symplectic Euler and implicit midpoint methods. Shown are plots for the values $h \in \{0.8, 0.2, 0.05\}$.

Appendix D. Supplementary plots of trajectory predictions

For completeness, we also provide two exemplary plots of just one coordinate of trajectories predicted by (S)HNNs, in comparison with the true trajectory (see Fig. D.6). This explains the origins of the mean squared L^2 error (MSE) as shown in Fig. 4 for the pendulum problem and (S)HNNs trained with $h = 0.8$: There is a progressive dephasing between the model's prediction and the ground truth, which is why the MSE also oscillates on a large scale. In fact, the total phase difference is continuously growing with larger time (yet, it is only represented modulo 2π in the plots C of Fig. 4).

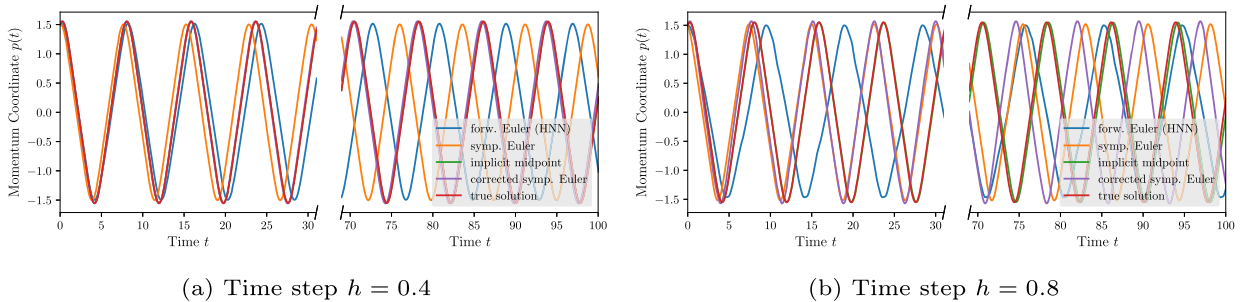


Fig. D.6. Momentum coordinate of predicted trajectories of the pendulum system, with $h = 0.4, 0.8$, starting from an initial position $p_0 = 1.5$ and $q_0 = -0.4$. In both plots, the prediction using the implicit midpoint method is almost completely hidden behind the true solution.

References

- [1] S. Greydanus, M. Dzamba, J. Yosinski, Hamiltonian neural networks, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019, <https://proceedings.neurips.cc/paper/2019/file/26cd8ecadce0d4efd6cc8a8725cbd1f8-Paper.pdf>.
- [2] T. Beucier, M. Pritchard, S. Rasp, J. Ott, P. Baldi, P. Gentine, Enforcing analytic constraints in neural networks emulating physical systems, *Phys. Rev. Lett.* 126 (2021) 098302, <https://doi.org/10.1103/PhysRevLett.126.098302>, <https://link.aps.org/doi/10.1103/PhysRevLett.126.098302>.
- [3] E. de Bezenac, A. Pajot, P. Gallinari, Deep learning for physical processes: incorporating prior scientific knowledge, in: *International Conference on Learning Representations*, 2018, <https://openreview.net/forum?id=By4HsfWAZ>.
- [4] S.L. Brunton, B.R. Noack, P. Koumoutsakos, Machine learning for fluid mechanics, *Annu. Rev. Fluid Mech.* 52 (1) (2020) 477–508, <https://doi.org/10.1146/annurev-fluid-010719-060214>.
- [5] S. Wiewel, M. Becher, N. Thuerey, Latent space physics: towards learning the temporal evolution of fluid flow, *Comput. Graph. Forum* 38 (2) (2019) 71–82, <https://doi.org/10.1111/cgf.13620>, <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13620>.
- [6] J.M. Sellier, G.M. Caron, J. Leygonie, Signed particles and neural networks, towards efficient simulations of quantum systems, *J. Comput. Phys.* 387 (2019) 154–162, <https://doi.org/10.1016/j.jcp.2019.02.036>, <https://www.sciencedirect.com/science/article/pii/S0021999119301536>.
- [7] E. Camporeale, The challenge of machine learning in space weather: nowcasting and forecasting, *Space Weather* 17 (8) (2019) 1166–1207, <https://doi.org/10.1029/2018SW002061>.
- [8] E. Camporeale, S. Wing, J. Johnson, *Machine Learning Techniques for Space Weather*, Elsevier Science, ISBN 9780128117897, 2018, <https://books.google.fr/books?id=zaRBDwAAQBAJ>.
- [9] M. Reichstein, G. Camps-Valls, B. Stevens, M. Jung, J. Denzler, N. Carvalhais, et al., Deep learning and process understanding for data-driven Earth system science, *Nature* 566 (7743) (2019) 195–204, <https://doi.org/10.1038/s41586-019-0912-1>, <https://www.nature.com/articles/s41586-019-0912-1>, number: 7743, Publisher: Nature Publishing Group.
- [10] J. Willard, X. Jia, S. Xu, M. Steinbach, V. Kumar, Integrating physics-based modeling with machine learning: a survey, *arXiv:2003.04919*, 2020.
- [11] S.L. Brunton, J.L. Proctor, J.N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, *Proc. Natl. Acad. Sci.* 113 (15) (2016) 3932–3937, <https://doi.org/10.1073/pnas.1517384113>, <https://www.pnas.org/content/113/15/3932>.
- [12] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D.K. Duvenaud, Neural ordinary differential equations, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, vol. 31, Curran Associates, Inc., 2018, <https://proceedings.neurips.cc/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf>.
- [13] D. Nguyen, S. Ouala, L. Drumetz, R. Fablet, Em-like learning chaotic dynamics from noisy and partial observations, *arXiv:1903.10335*, 2019.
- [14] F. Regazzoni, L. Dedé, A. Quarteroni, Machine learning for fast and reliable solution of time-dependent differential equations, *J. Comput. Phys.* 397 (2019) 108852, <https://doi.org/10.1016/j.jcp.2019.07.050>, <https://www.sciencedirect.com/science/article/pii/S0021999119305364>.
- [15] W.R. Hamilton, On a general method in dynamics; by which the study of the motions of all free systems of attracting or repelling points is reduced to the search and differentiation of one central relation, or characteristic function, *Philos. Trans. R. Soc. Lond.* 124 (1834) 247–308, <http://www.jstor.org/stable/108066>.
- [16] L.N. Hand, J.D. Finch, *Analytical Mechanics*, Cambridge University Press, 1998.
- [17] T. Bertalan, F. Dietrich, I. Mezić, I.G. Kevrekidis, On learning hamiltonian systems from data. *Chaos: an interdisciplinary, J. Nonlinear Sci.* 29 (12) (2019) 121107, <https://doi.org/10.1063/1.5128231>.
- [18] P. Toth, D.J. Rezende, A. Jaegle, S. Racanière, A. Botev, I. Higgins, Hamiltonian generative networks, in: *International Conference on Learning Representations*, 2020, <https://openreview.net/forum?id=HJenn6VFvB>.
- [19] Z. Chen, J. Zhang, M. Arjovsky, L. Bottou, Symplectic recurrent neural networks, in: *International Conference on Learning Representations*, 2020, <https://openreview.net/forum?id=BkgYPREtPr>.
- [20] Y.D. Zhong, B. Dey, A. Chakraborty, Symplectic ODE-Net: learning hamiltonian dynamics with control, in: *International Conference on Learning Representations*, 2020, <https://openreview.net/forum?id=ryxmb1rKDS>.
- [21] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, S. Ho, Lagrangian neural networks, in: *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020, <https://openreview.net/forum?id=iE8tF4Nq>.
- [22] A. Zhu, P. Jin, Y. Tang, Deep hamiltonian networks based on symplectic integrators, *arXiv:2004.13830*, 2020.
- [23] S. Xiong, Y. Tong, X. He, S. Yang, C. Yang, B. Zhu, Nonseparable symplectic neural networks, in: *International Conference on Learning Representations*, 2021, <https://openreview.net/forum?id=B5VvQrI49Pa>.
- [24] D. DiPietro, S. Xiong, B. Zhu, Sparse symplectically integrated neural networks, in: H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 6074–6085, <https://proceedings.neurips.cc/paper/2020/file/439fca360bc99c315c5882c4432ae7a4-Paper.pdf>.
- [25] P. Jin, Z. Zhang, A. Zhu, Y. Tang, G.E. Karniadakis, SympNets: intrinsic structure-preserving symplectic networks for identifying Hamiltonian systems, *Neural Netw.* 132 (2020) 166–179, <https://doi.org/10.1016/j.neunet.2020.08.017>, <https://www.sciencedirect.com/science/article/pii/S0893608020303063>.
- [26] Y. Tong, S. Xiong, X. He, G. Pan, B. Zhu, Symplectic neural networks in Taylor series form for hamiltonian systems, *J. Comput. Phys.* 437 (2021) 110325, <https://doi.org/10.1016/j.jcp.2021.110325>, <https://www.sciencedirect.com/science/article/pii/S0021999121002205>.
- [27] R. Chen, M. Tao, Data-driven prediction of general hamiltonian dynamics via learning exactly-symplectic maps, in: M. Meila, T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, vol. 139, PMLR, 2021, pp. 1717–1727, <https://proceedings.mlr.press/v139/chen21r.html>.
- [28] E. Hairer, C. Lubich, G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, Springer, 2006.
- [29] P. Chartier, E. Hairer, G. Vilmart, Numerical integrators based on modified differential equations, *Math. Comput.* 76 (2007) 1941–1953, <https://doi.org/10.1090/S0025-5718-07-01967-9>.
- [30] J. Lu, Z. Shen, H. Yang, S. Zhang, Deep network approximation for smooth functions, *SIAM J. Math. Anal.* 53 (5) (2021) 5465–5506, <https://doi.org/10.1137/20M134695X>.
- [31] T. De Ryck, S. Lanthaler, S. Mishra, On the approximation of functions by tanh neural networks, *Neural Netw.* 143 (2021) 732–750, <https://doi.org/10.1016/j.neunet.2021.08.015>, <https://www.sciencedirect.com/science/article/pii/S0893608021003208>.
- [32] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), *3rd International Conference on Learning Representations (ICLR)*, 2015, San Diego, CA, USA, <http://arxiv.org/abs/1412.6980>.
- [33] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: *7th International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, *OpenReview.net*, 2019, <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [34] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., Pytorch: an imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035, <http://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [35] Nvidia Corporation, CUDA toolkit documentation, <https://docs.nvidia.com/cuda/archive/11.2.0/index.html>, 2020.

- [36] J.R. Dormand, P.J. Prince, A family of embedded Runge-Kutta formulae, *J. Comput. Appl. Math.* 6 (1) (1980) 19–26, [https://doi.org/10.1016/0771-050X\(80\)90013-3](https://doi.org/10.1016/0771-050X(80)90013-3), <https://www.sciencedirect.com/science/article/pii/0771050X80900133>.
- [37] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nat. Methods* 17 (2020) 261–272, <https://doi.org/10.1038/s41592-019-0686-2>.
- [38] M. Tao, Explicit symplectic approximation of nonseparable hamiltonians: algorithm and long time performance, *Phys. Rev. E* 94 (2016) 043303, <https://doi.org/10.1103/PhysRevE.94.043303>, <https://link.aps.org/doi/10.1103/PhysRevE.94.043303>.